

Edge cases (a bad release that slips past the gate)

— Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Every approach below starts by proving the bypass is real, then fixes it a different way.

Basic Version

Approach 1 — prove the bypass, then a plain hash check

```
# release_gate_practice.py – Edge cases section
import hashlib
import json

# --- step 1: prove the bypass is real ---
with open("live_version.txt", "w") as f:
    f.write("v1")    # v1 never passed the 0.9 threshold in release_gate – nothing

with open("live_version.txt") as f:
    print("live (before any fix):", f.read())
```

Expected output:

```
live (before any fix): v1
```

Nothing checked whether `v1` actually passed the gate — it's just a text file, and text files don't know the difference.

```
# release_gate_practice.py – Edge cases section
# --- step 2: a plain hash check ---
def deploy_and_save_pointer(version_name):
    # imagine this only runs after deploy()'s score check passed
    check = hashlib.sha256(version_name.encode()).hexdigest()
    with open("live_version.json", "w") as f:
        json.dump({"version_name": version_name, "check": check}, f)
```

```
def get_live_version():
    with open("live_version.json") as f:
        pointer = json.load(f)
        expected = hashlib.sha256(pointer["version_name"].encode()).hexdigest()
        if pointer["check"] != expected:
            print("Warning: live pointer looks tampered with!")
            return None
        return pointer["version_name"]

deploy_and_save_pointer("v2")
print(get_live_version())
```

Expected output:

```
v2
```

Now hand-edit `live_version.json`, changing `"version_name"` to `"v1"` but leaving `"check"` as it was:

```
print(get_live_version())
```

Expected output:

```
Warning: live pointer looks tampered with!
None
```

This version works correctly for what the exercise asks — a careless hand-edit is now caught. It's not safe against someone who reads this same script, though:

`hashlib.sha256(version_name.encode())` uses no secret, so anyone who can see this code can recompute a matching `"check"` for any version name they like. That's what Intermediate fixes.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `hmac`, signed with a secret

```
# release_gate_practice.py – Edge cases section
import hmac
import hashlib
import json

SECRET = "replace-with-a-real-secret-not-committed-to-git"

def sign(version_name: str) -> str:
    return hmac.new(SECRET.encode(), version_name.encode(), hashlib.sha256).hexdigest()

def save_pointer(version_name: str) -> None:
    pointer = {"version_name": version_name, "signature": sign(version_name)}
    with open("live_version.json", "w") as f:
        json.dump(pointer, f)

def get_live_version() -> str | None:
    with open("live_version.json") as f:
        pointer = json.load(f)
        expected = sign(pointer["version_name"])
        if not hmac.compare_digest(pointer["signature"], expected):
            print("Warning: live pointer looks tampered with!")
            return None
        return pointer["version_name"]

save_pointer("v2")
print(get_live_version())
```

Expected output:

v2

Hand-edit `live_version.json`'s `"version_name"` to `"v1"` without recomputing `"signature"`:

```
print(get_live_version())
```

Expected output:

```
Warning: live pointer looks tampered with!  
None
```

Unlike Basic, an attacker who has read this script still can't produce a matching `"signature"` for `"v1"` without also knowing `SECRET` — `hmac.new` ties the signature to both the version name *and* a value that isn't written down inside `live_version.json` itself.

Difference from Basic: Basic's plain `sha256` hash catches a careless hand-edit but is trivially fakeable by anyone who's read the code, since nothing about it is secret. Intermediate's `hmac.new(SECRET.encode(), ...)` depends on a value that lives only in `deploy_gate.py` (or, in a real project, an environment variable) — never inside the pointer file that gets edited — so recomputing a valid signature requires knowing that secret, not just the hashing algorithm. `hmac.compare_digest` also replaces a plain `==`, avoiding a timing side-channel that a naive string comparison has (a minor point for a local file, a real one for anything checked over a network).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — remove the separate pointer file; the version log is the only source of truth

```
# release_gate_practice.py – Edge cases section
import json
import time
from datetime import datetime, timezone
from pathlib import Path

def load_versions(path: str = "versions.json") -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def save_versions(versions: list[dict], path: str = "versions.json") -> None:
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)

def save_version(name: str, prompt_text: str, path: str = "versions.json") -> None:
    versions = load_versions(path)
    versions.append({
        "version_name": name,
        "prompt_text": prompt_text,
        "created_at": datetime.now(timezone.utc).isoformat(),
        "passed_gate": False,
    })
    save_versions(versions, path)

def mark_passed(version_name: str, path: str = "versions.json") -> None:
    versions = load_versions(path)
    for v in versions:
        if v["version_name"] == version_name:
            v["passed_gate"] = True
    save_versions(versions, path)

def get_live_version(path: str = "versions.json") -> str | None:
    versions = load_versions(path)
    passing = []
```

```

    for v in versions:
        if v.get("passed_gate"):
            passing.append(v)

    if not passing:
        return None

    latest = passing[0]
    for v in passing:
        if v["created_at"] > latest["created_at"]:
            latest = v
    return latest["version_name"]

save_version("v1", "Answer the customer's question.")
time.sleep(0.01)
save_version("v2", "Answer the customer's question politely and cite the source.")

print("live before anything passes:", get_live_version())

mark_passed("v2")  # imagine this only runs inside deploy(), after the score check
print("live after v2 passes:", get_live_version())

```

Expected output:

```

live before anything passes: None
live after v2 passes: v2

```

There's no `live_version.txt` or `live_version.json` left at all — `get_live_version()` derives the answer straight from the version log's own `passed_gate` field, the exact same record `deploy()` already writes.

Approach 2 — showing what a bypass attempt now actually requires

```

# release_gate_practice.py – Edge cases section
# an attacker (or a careless teammate) hand-edits versions.json directly,
# setting v1's passed_gate to True too – this is still *possible*, since
# nothing here is cryptographically signed
versions = load_versions()
for v in versions:
    if v["version_name"] == "v1":
        v["passed_gate"] = True
save_versions(versions)

print("live after hand-editing v1's passed_gate:", get_live_version())

```

Expected output:

```
live after hand-editing v1's passed_gate: v2
```

`v2` is still what comes back — `get_live_version()` picks the *most recent* passing entry, and `v2` was saved after `v1`. The hand-edit "worked" in the sense that `v1` now also shows `passed_gate: true`, which is a real problem worth noticing (this log entry is no longer trustworthy evidence that `v1` was ever actually tested) — but it no longer silently flips what the rest of the system treats as live, the way editing a separate `live_version.txt` used to. Combine this with Intermediate's `hmac` signature (signing each version's `passed_gate` claim, not just a separate pointer file) to make even this deeper edit detectable — left as a natural next step past this exercise.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate makes a separate pointer file's claim ("this is what's live") cryptographically checkable. Approach 1 asks a more basic design question first — does a separate pointer file need to exist at all? — and removes it, making "what's live" a value *derived* from the version log every time, not a second piece of state that can drift from the truth or be edited on its own. Approach 2 is honest about the limit of Approach 1 alone: without also signing the log's own `passed_gate` field, a determined hand-edit to the log itself is still possible — the win is that there's no longer a separate, easy-to-miss side door; the log is the only door, and it's the one door everything else already watches.

Which one should you actually write? Intermediate's `hmac` signature is the right minimum for a real project — cheap, no design change required, and it turns "silently wrong" into "loudly detected." Advanced Approach 1 is worth doing as well once you notice a separate pointer file existing purely as a target for this exact exercise's failure mode — collapsing it into the version log removes an entire class of "these two records disagree" bugs, not just the deliberate-bypass one. For the Build Task specifically, combine both: `passed_gate` lives on the version log entry (Advanced), and that field only gets set by code that also checks an `hmac` signature computed from something outside the log itself (Intermediate) — so both "which file do I trust" and "can this field be faked" are answered.